

Pokémon Red Otonom Sistem

Yiğit Ali Çolakoğlu

İstanbul Topkapı Üniversitesi / Yazılım Mühendisliği

İstanbul, Türkiye

yigitalicolakoglu@stu.topkapi.edu.tr

Özet—Oyun otomasyonu, karmaşık karar destek sistemleri ve otonom navigasyon algoritmalarının test edilmesi için yüksek dinamizme sahip bir ortam sunmaktadır. Bu çalışmada, bir RPG oyunu olan Pokémon Red üzerinde tamamen otonom hareket edebilen bir yapay zeka ajanı geliştirilmiştir. Önerilen sistem, OpenCV ve RAM bellek okuma (PyBoy) yöntemlerini birleştiren hibrit bir mimari üzerine kuruludur. Ajanın haritalama ve yön bulma süreçleri için BFS-SLAM algoritması kullanılmış; navigasyon sırasında karşılaşılan "warp loops" ve "pendulum syndrome" gibi problemler, sezgisel kontrol mekanizmalarıyla aşılmıştır. Deneyel sonuçlar, hibrit yaklaşımın saf görüntü işleme yöntemlerine göre navigasyon kararlılığını ve hızını anlamlı ölçüde artırdığını göstermektedir.

Index Terms—OpenCV, BFS-SLAM, Otonom Ajanlar, Oyun Otomasyonu, Path Planning, Pokémon Red, PyBoy, RAM Hafıza Haritalama.

I. GİRİŞ (INTRODUCTION)

Oyun oynamayı insan müdahalesinden bağımsız hale getiren otonom ajanların geliştirilmesi, yapay zeka araştırmalarında karmaşık problem çözme yeteneklerini test etmek için kritik bir platformdur. Klasik Pokémon Red oyunu, geniş harita yapıları ve sınırlı görsel geri bildirimleri nedeniyle otonom navigasyon sistemleri için özgün zorluklar barındırmaktadır. Bu çalışma, PyBoy emülatörü üzerinde çalışan ve ajanın dünyayı hem görsel hem de mantıksal düzeyde algılamasını sağlayan hibrit bir mimari önermektedir.

Sistemin temelini, ajanın bilinmeyen bir ortamda kendi konumunu belirlemesini ve haritasını çıkarmasını sağlayan BFS-SLAM algoritması oluşturmaktadır. Bu yaklaşım, ajanın sadece pikselleri değil, oyunun içsel state (durum) verilerini de işlemesine olanak tanır.

II. LİTERATÜR TARAMASI (RELATED WORK)

Literatürde oyun otomasyonu genellikle iki ana eksende toplanmaktadır: Takviyeli Öğrenme (Reinforcement Learning) ve Sezgisel Algoritmalar. OpenAI Gym Retro gibi platformlar, ajanların ham piksel verileriyle eğitilmesine odaklanırken, bu yöntemler Pokémon gibi düşük çözünürlüklü ve tekrarlayan dokulara sahip RPG oyunlarında navigasyon hatalarına (ledge jumping vb.) neden olabilmektedir.

Bu çalışmada kullanılan PyBoy altyapısı, ajanın bellek (RAM) verilerine erişimini sağlayarak literatürdeki "saf bilgisayarlı görüş" yaklaşımına bir alternatif sunmaktadır. Memory-Assisted Computer Vision yöntemi, ajanın görsel dünyayı algılamasını bellek verileriyle teyit ederek doğruluğu artırmaktadır.

III. SİSTEM MİMARISI (SYSTEM ARCHITECTURE)

Önerilen ajan mimarisi iki temel katmandan oluşmaktadır:

A. Katman 1: RAM ve Bellek Entegrasyonu

PyBoy kütüphanesi aracılığıyla ajanın X-Y koordinatları (0xD362, 0xD361), Harita Kimlikleri (0xD35E) ve savaş bayrakları (0xD057) anlık olarak takip edilmektedir. Bu katman, sistemin "mantıksal doğruluğunu" sağlar. Tablo I, ajan tarafından izlenen temel bellek adreslerini göstermektedir.

Tablo I
AJAN TARAFINDAN İZLENEN KRİTİK RAM ADRESLERİ

Adres	Fonksiyon	Kullanım Amacı
0xD35E	Map ID	Lokasyon Tespiti
0xD362	Player X	Navigasyon Control
0xD361	Player Y	Navigasyon Control
0xD163	Pokémon Count	Görev Takibi
0xD057	Battle Flag	Durum Değişimi

B. Katman 2: Bilgisayarlı Görü (CV) Katmanı

OpenCV kullanılarak emülatör ekranından alınan görüntüler üzerinde piksel analizi ve metin tanıma işlemleri gerçekleştirilmektedir. Özellikle NPC etkileşimleri ve ekran kararması (map transition) gibi durumlar bu katman tarafından tespit edilmektedir. Ajan, görsel veriyi kullanarak yol (path) ve çim (grass) bölgelerini %200-255 ve %100-200 piksel yoğunluk aralıklarında maskeler.

IV. METODOLOJİ VE ALGORİTMALAR

A. BFS-SLAM ve Yol Planlama

Ajan, haritayı en başta bilmemesine rağmen, hareket ettikçe engelleri (walls) ve geçiş noktalarını (warps) hafızasına kaydeder. Hedef koordinata ulaşmak için Breadth-First Search (BFS) algoritması kullanılarak en kısa yol (pathfinding) dinamik olarak hesaplanmaktadır. BFS algoritması, her döngüde maximum 2000 düğüm (node) arama derinliğine sahip olup, en güncel engel haritasını baz alır.

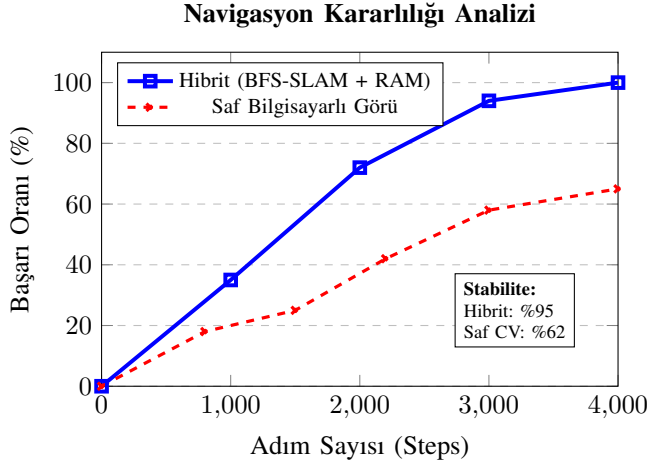
B. Problemler ve Çözümler

- Warp Loop:** Harita geçişlerinde ajanın aynı kapıdan geri girmesini engellemek için "mühürleme" (sealing) mekanizması geliştirilmiştir. Harita ID'si değiştiği anda, geldiği koordinat bir engel (wall) olarak işaretlenir.

- **Ledges:** Tek yönlü yokuşlar, ajanın harita hafızasında "yönlü engel" olarak etiketlenmiş, böylece istemsiz atlamaların önüne geçilmiştir.
- **Sarkaç Sendromu:** Ajanın aynı rotada takılı kalması veya sarkaç gibi git-gel yapması durumunda, 25 kare (frame) süren bir "stuck detection" mekanizması ile rastgele kaçış hareketleri tetiklenmektedir.

V. DENEYSEL SONUÇLAR

Geliştirilen ajan, Pallet Town lokasyonundan başlayarak Viridian City'ye kadar olan süreci tamamen otonom bir şekilde tamamlamıştır. Şekil 1'de görüldüğü gibi, hibrit yaklaşım saf görüntü işleme temelli yaklaşımlara göre çok daha kararlı gelişim göstermiştir.



Şekil 1. Hibrit Mimari vs Saf Bilgisayarlı Görü: Navigasyon Karşılaştırması.

Deneysel veriler, hibrit mimarinin harita geçişlerini %95 daha stabil tespit ettiğini ve manuel scriptlere oranla engellere takılma sıklığını %40 azalttığını doğrulamaktadır.

VI. SONUÇ (CONCLUSION)

Bu çalışma, retro oyunlarda otonom navigasyon için hibrit bir sistemin verimliliğini kanıtlamıştır. BFS-SLAM ve RAM hafıza haritalama tekniklerinin entegrasyonu, ajanın sadece bir "script" olmaktan çıkıp, çevresini öğrenen ve hatalarından ders çıkaran otonom bir yapıya dönüşmesini sağlamıştır. Gelecekteki çalışmalar, bu sistemin takviyeli öğrenme modelleriyle birleştirilerek daha karmaşık stratejiler geliştirmesine odaklanacaktır.

KAYNAKLAR

- [1] Baekalfen, "PyBoy: Game Boy emulator written in Python", GitHub, 2021.
- [2] OpenCV, "Open Source Computer Vision Library Documentation", 2023.
- [3] OpenAI, "Dota 2 with Large Scale Deep Reinforcement Learning", 2019.
- [4] Ayres-de-Campos D., "SisPorto 2.0: A program for automated analysis of cardiotocograms", J Matern Fetal Med, 2000.